

DONCIEL TM

Journal de Trading Professionnel

Guide de Maintenance

Elements a conserver & Mode d'emploi de mise a jour

Mars 2026

Version 1.0 | Document confidentiel

Table des Matieres

PARTIE 1 Elements a Conserver

1.1 Comptes et Acces

1.2 Variables d'Environnement (.env)

1.3 Architecture Technique

1.4 Structure du Projet

1.5 Fonctionnalites de l'Application

1.6 Fichiers Critiques

1.7 Modeles de Base de Donnees (Prisma)

1.8 Dependances Cles

PARTIE 2 Mode d'Emploi - Mise a Jour

2.1 Mise a jour du code source

2.2 Mise a jour des dependances

2.3 Mise a jour de la base de donnees

2.4 Deploiement sur Vercel

2.5 Verification apres deploiement

2.6 Sauvegarde de la base de donnees

2.7 En cas de probleme

2.8 Checklist de maintenance

PARTIE 1

Elements a Conserver

1.1 Comptes et Acces

Les comptes suivants sont essentiels au fonctionnement de l'application DONCIEL TM. Conservez ces informations en lieu sur et assurez-vous que les acces restent valides.

Service	Identifiant / URL	Role	Priorite
GitHub	Prince-ilunga/donciel-tm	Depot de code source	Critique
Vercel	https://donciel-trading.vercel.app	Hebergement & deploiement	Critique
Neon PostgreSQL	Connection string dans .env	Base de donnees hebergee	Critique
Cloudinary	Cloud name, API key & secret dans .env	Stockage medias (captures d'ecran, videos)	Important
JWT / NextAuth	NEXTAUTH_SECRET dans .env	Securite d'authentification	Critique

Attention

Ne partagez jamais les cles d'API ou les secrets JWT publiquement. Utilisez toujours le tableau de bord Vercel pour configurer les variables d'environnement en production.

1.2 Variables d'Environnement (.env)

Le fichier `.env` contient toutes les configurations sensibles de l'application. Ces variables doivent etre definies a la fois localement et sur Vercel.

Variable	Description	Exemple	Obligatoire
<code>DATABASE_URL</code>	URL de connexion Neon PostgreSQL	<code>postgresql://user:pass@ep-xxx.neon.tech/db</code>	Oui
<code>NEXTAUTH_SECRET</code>	Secret pour le chiffrement JWT NextAuth	<code>votre-secret-aleatoire</code>	Oui
<code>CLOUDINARY_CLOUD_NAME</code>	Nom du cloud Cloudinary	<code>donciel-tm</code>	Prod
<code>CLOUDINARY_API_KEY</code>	Cle API Cloudinary	<code>123456789012345</code>	Prod
<code>CLOUDINARY_API_SECRET</code>	Secret API Cloudinary	<code>abcdefg...</code>	Prod

Note

En developpement local, `DATABASE_URL` peut pointer vers un fichier SQLite local (ex: `file:/home/z/my-project/db/custom.db`). En production sur Vercel, elle doit pointer vers l'instance Neon PostgreSQL.

1.3 Architecture Technique

L'application est construite avec les technologies modernes suivantes :

Composant	Technologie	Version
Framework	Next.js (App Router)	16.x
Langage	TypeScript	5.x
Base de donnees	Prisma ORM + Neon PostgreSQL	Prisma 6.x
Stockage medias	Cloudinary	2.x
Deploiement	Vercel	—

Composant	Technologie	Version
UI Components	shadcn/ui + Radix UI	Derniere
Style CSS	Tailwind CSS	4.x
State Management	Zustand	5.x
Authentification	NextAuth.js (JWT + bcryptjs)	4.x
Graphiques	Recharts	2.x
Runtime	Bun	—
React	React 19	19.x

1.4 Structure du Projet

L'arborescence du projet suit la convention Next.js App Router :

```
donciel-tm/ src/ app/ – Pages et API routes (App Router) api/ – Endpoints API (auth, trades, upload, videos, etc.) components/ – Composants React réutilisables layout/ – Navigation, sidebar, header ui/ – Composants shadcn/ui stores/ – Etat global Zustand lib/ – Utilitaires (auth, db, i18n, cloundinary) prisma/ – Schema de base de données public/ – Fichiers statiques package.json – Dependances et scripts .env – Variables d'environnement (NE PAS COMMITTER) next.config.ts – Configuration Next.js tailwind.config.ts – Configuration Tailwind CSS
```

1.5 Fonctionnalités de l'Application

DONCIEL TM offre les fonctionnalités suivantes :

Execution DONCIEL

Phases de progression (Phase Initiale, Phase 1, Phase 2) avec cycles valides et RR cumule.

Setup

Gestion des données DONCIEL et trades personnalisés. Saisie, vérification et filtres.

Journal de Trading

Calendrier interactif pour visualiser les trades par date. Détails complets de chaque trade.

Distribution RR

Graphiques d'analyse Risk-Reward : par direction, setup, session, market condition, timeframe, actif.

Timing Analysis

Performance temporelle : par jour, heure, mois, session. Graphiques en barres et courbes.

Videos du Setup

Catégories : Structure du Marché, Biais, Zones de Valeur, Modèles d'Entrées, Setups.

Notes de Préparation

Notes journalières, hebdomadaires et mensuelles pour la préparation de trading.

Gestion des Roles

Admin : vérification des membres, approbation/rejet. Statistiques individuelles et globales.

Export PDF

Export des données et des trades en format PDF pour archivage.

Mode Sombre / Clair

Interface adaptable avec thème sombre et clair. Basculement instantané.

Internationalisation FR/EN

Interface complètement traduite en français et anglais.

Captures d'ecran

Upload de captures d'ecran (analyse, entree, sortie) pour chaque trade via Cloudinary.

1.6 Fichiers Critiques

Les fichiers suivants sont essentiels au bon fonctionnement de l'application. Toute modification doit être effectuée avec précaution.

Fichier	Role	Action si perdu
<code>prisma/schema.prisma</code>	Structure de la base de données (modèles User, Trade, Screenshot, Video, Note, CustomPair, UploadSession)	Restaurer depuis Git
<code>src/lib/auth.ts</code>	Authentification JWT (login, register, session)	Restaurer depuis Git
<code>src/lib/db.ts</code>	Client Prisma (connexion à la base de données)	Restaurer depuis Git
<code>src/lib/i18n.ts</code>	Traductions FR/EN complètes (500+ clés)	Restaurer depuis Git
<code>src/components/layout/main-app.tsx</code>	Navigation principale et structure de l'application	Restaurer depuis Git
<code>src/stores/app-store.ts</code>	État global Zustand (onglet actif, langue, utilisateur)	Restaurer depuis Git
<code>.env</code>	Variables d'environnement (secrets)	Reconfigurer manuellement
<code>package.json</code>	Liste des dépendances et scripts	Restaurer depuis Git

Danger

Le fichier `.env` n'est JAMAIS commité dans Git. En cas de perte, les secrets doivent être régénérés depuis les consoles respectives (Neon, Cloudinary, etc.).

1.7 Modèles de Base de Données (Prisma)

Le schéma Prisma définit les modèles suivants dans la base PostgreSQL :

Modele	Description	Champs cles
User	Utilisateurs de l'application	email, password, name, role (admin/user), status (pending/approved/rejected), language (fr/en)
Trade	Enregistrements de trades	date, pair, direction, session, marketCondition, timeframe, setup, structure, entryModel, entryPrice, stopLoss, takeProfit, exitPrice, rr, pnl, result, emotions, confluence, mistakes, lessons
Screenshot	Captures d'ecran des trades	type (analysis/entry/exit), url
Video	Videos educatives	category (STRUCTURE/BIAIS/ZONES/MODELS/SETUPS), title, description, url
Note	Notes de preparation	type (DAY/WEEK/MONTH), title, content, date
CustomPair	Paires de devises personnalisees	name
UploadSession	Sessions d'upload chunked	uploadId, filename, totalSize, totalChunks, category, receivedChunks

Relations

Un `User` possede plusieurs `Trade` , `Note` , `CustomPair` et `Video` . Un `Trade` possede plusieurs `Screenshot` . La suppression d'un utilisateur entraine la suppression en cascade de toutes ses donnees associees.

1.8 Dependances Cles

Les dependances principales du projet (extrait du `package.json`) :

Dependances de Production

Package	Version	Usage
next	^16.1.1	Framework principal
react / react-dom	^19.0.0	Bibliotheque UI
@prisma/client	^6.11.1	ORM base de donnees
prisma	^6.11.1	CLI Prisma
next-auth	^4.24.11	Authentification
bcryptjs	^3.0.3	Hachage mots de passe
jose	^6.2.3	JWT signing/verification
cloudinary	^2.10.0	Stockage medias
zustand	^5.0.6	State management
recharts	^2.15.4	Graphiques
@tanstack/react-query	^5.82.0	Requetes asynchrones
@tanstack/react-table	^8.21.3	Tableaux de donnees
jspdf / jspdf-autotable	^4.2.1 / ^5.0.8	Generation PDF
pdf-lib	^1.17.1	Manipulation PDF
date-fns	^4.1.0	Manipulation de dates
zod	^4.0.2	Validation de schemas
react-hook-form	^7.60.0	Gestion de formulaires
framer-motion	^12.23.2	Animations
lucide-react	^0.525.0	Icones
next-themes	^0.4.6	Gestion themes sombre/clair
next-intl	^4.3.4	Internationalisation

Package	Version	Usage
sonner	^2.0.6	Notifications toast
sharp	^0.34.3	Traitement d'images

Dependances de Developpement

Package	Version	Usage
tailwindcss	^4	Framework CSS
@tailwindcss/postcss	^4	Plugin PostCSS Tailwind
typescript	^5	Compilateur TypeScript
eslint / eslint-config-next	^9 / ^16.1.1	Linting
bun-types	^1.3.4	Types Bun

PARTIE 2

Mode d'Emploi — Mise a Jour

2.1 Mise a jour du code source

Pour recuperer les dernieres modifications du depot GitHub :

```
$ git pull origin main
```

Bonne pratique

Avant de tirer les modifications, assurez-vous d'avoir commite vos propres changements avec `git add .` et `git commit -m "message"` pour eviter les conflits.

2.2 Mise a jour des dependances

Apres avoir mis a jour le code source, installez les eventuelles nouvelles dependances :

```
$ bun install
```

Cela mettra a jour le dossier `node_modules` selon le fichier `package.json` et `bun.lock`.

Scripts disponibles

Commande	Description
<code>bun run dev</code>	Lance le serveur de developpement sur le port 3000
<code>bun run build</code>	Genere le client Prisma puis compile le projet
<code>bun run start</code>	Lance le serveur de production
<code>bun run lint</code>	Verifie la qualite du code avec ESLint

2.3 Mise a jour de la base de donnees

Si le schema Prisma (`prisma/schema.prisma`) a ete modifie, vous devez synchroniser la base de donnees :

```
$ bun run db:push # ou equivalentement: bunx prisma db push
```

Commandes Prisma utiles

Commande	Description
<code>bun run db:push</code>	Pousse le schema vers la DB (sans migration)
<code>bun run db:generate</code>	Regenere le client Prisma
<code>bun run db:migrate</code>	Cree et applique une migration
<code>bun run db:reset</code>	Reinitialise la base de donnees (dangereux !)

Danger

La commande `db:reset` supprime toutes les donnees de la base. Ne l'utilisez qu'en cas de reinitialisation complete. Effectuez toujours une sauvegarde avant.

2.4 Deploiement sur Vercel

Le deploiement est automatique : chaque push sur la branche `main` declenche un nouveau deploiement sur Vercel.

```
$ git push origin main
```

- 1 Effectuez vos modifications en local et testez avec `bun run dev`
- 2 Commitez vos changements : `git add .` puis `git commit -m "description"`
- 3 Poussez sur GitHub : `git push origin main`
- 4 Vercel detecte le push et lance le build automatiquement
- 5 Une fois le build termine, le site est mis a jour sur `donciel-trading.vercel.app`

Zero-downtime

Vercel deploie les nouvelles versions sans interruption de service. L'ancienne version reste active jusqu'a ce que la nouvelle soit prete.

2.5 Verification apres deploiement

Apres chaque deploiement, verifiez les points suivants :

- 1 Visitez `https://donciel-trading.vercel.app` et verifiez que la page d'accueil se charge
- 2 Connectez-vous avec un compte existant et verifiez l'authentification
- 3 Consultez le dashboard Vercel pour verifier les logs de build et d'execution
- 4 Testez les fonctionnalites cles : ajout de trade, journal, graphiques
- 5 Verifiez que l'upload de captures d'ecran fonctionne (Cloudinary)

Dashboard Vercel

Accédez au dashboard a l'adresse `https://vercel.com/dashboard` pour voir les deploiements, les logs en temps reel et les erreurs eventuelles.

2.6 Sauvegarde de la base de donnees

Il est recommande d'effectuer des sauvegardes regulieres de la base de donnees Neon PostgreSQL.

Option 1 : Neon Console

1. Connectez-vous a `https://console.neon.tech`
2. Selectionnez votre projet
3. Utilisez la fonctionnalite d'export ou de branch pour creer une sauvegarde

Option 2 : pg_dump en ligne de commande

```
# Sauvegarde complete de la base $ pg_dump "postgresql://user:pass@ep-xxx.neon.tech/dbname" > backup_$(date +%Y%m%d).sql # Restauration $ psql "postgresql://user:pass@ep-xxx.neon.tech/dbname" < backup_20260304.sql
```

Recommandation

Effectuez une sauvegarde au moins une fois par semaine, et systematiquement avant toute modification du schema de base de donnees.

2.7 En cas de probleme

Guide de resolution des problemes courants :

Le site ne se charge pas

- Verifiez le statut du deploiement dans le dashboard Vercel
- Consultez les logs d'erreur dans la section *Functions* de Vercel
- Verifiez que les variables d'environnement sont correctes sur Vercel

Erreur de base de donnees

- Verifiez que `DATABASE_URL` est correct dans les variables Vercel
- Assurez-vous que le schema Prisma est synchronise : `bun run db:push`
- Verifiez que la base Neon n'est pas en pause (Neon met en pause les bases inactives)

Les uploads ne fonctionnent pas

- Verifiez les variables `CLOUDINARY_CLOUD_NAME` , `CLOUDINARY_API_KEY` , `CLOUDINARY_API_SECRET`
- Testez les identifiants dans la console Cloudinary
- Verifiez les limites de stockage de votre plan Cloudinary

Erreur d'authentification

- Verifiez que `NEXTAUTH_SECRET` est defini sur Vercel
- Regenez le secret si necessaire et mettez a jour partout
- Verifiez que la base de donnees contient bien les utilisateurs

Erreur de build Vercel

- Lisez attentivement les logs de build dans le dashboard Vercel
- Verifiez que `bun run build` fonctionne en local sans erreur
- Assurez-vous que `prisma generate` est execute avant le build (il est dans le script `build`)

Ressources utiles

- Documentation Vercel : <https://vercel.com/docs>
- Documentation Prisma : <https://www.prisma.io/docs>
- Documentation Neon : <https://neon.tech/docs>
- Documentation Cloudinary : <https://cloudinary.com/documentation>

2.8 Checklist de Maintenance

Utilisez cette checklist pour chaque intervention de maintenance :

#	Tache	Commande / Action
1	Sauvegarder la base de donnees	<code>pg_dump</code> ou Neon Console
2	Recuperer le dernier code	<code>git pull origin main</code>
3	Installer les dependances	<code>bun install</code>
4	Synchroniser la base de donnees	<code>bun run db:push</code> (si schema modifie)
5	Tester en local	<code>bun run dev</code> puis tests manuels
6	Deployer en production	<code>git push origin main</code>
7	Verifier le deploiement	Visiter <code>donciel-trading.vercel.app</code>
8	Verifier les logs Vercel	Dashboard Vercel > Logs
9	Confirmer les fonctionnalites	Tests manuels des fonctionnalites cles

Fin du Guide de Maintenance DONCIEL TM

Document genere le 4 mars 2026 | Version 1.0

Pour toute question, consultez le depot GitHub : Prince-ilunga/donciel-tm